Guide Vast.ai : allocation de la VM et gestion du crédit

Introduction

Une RTX 3060 (12 Go) en On-Demand, à environ 0,083 $/h, suffit pour tout le projet NeuroVision AI Part 2 : 24 h d'allumage continu coûtent environ 2 $.

Ce guide explique aux apprenants comment choisir et allouer une machine sur Vast.ai, comprendre les types de location, lire une offre et ne pas gaspiller leur crédit. Le projet fait tourner sur la même instance MobileNetV2 (transfer learning et fine-tuning), MLflow, Streamlit et, en bonus, Airflow.

Les prix de Vast.ai sont ceux d'une place de marché : ils changent chaque jour. Les montants ci-dessous sont des ordres de grandeur relevés le 19 octobre 2026.

Comment le crédit est consommé

Le crédit baisse dès que l'instance est allumée, que le GPU calcule ou non : la facturation se fait à la seconde, au tarif affiché (par exemple 0,083 $/h). Ouvrir Jupyter, utiliser le token ou lancer un entraînement ne change pas le prix.

État de l'instance	GPU facturé	Stockage facturé	À retenir
Running (allumée)	Oui	Oui	Facturation continue, même au repos
Stopped (arrêtée)	Non	Oui	Quelques centimes par jour, les données sont conservées
Destroyed (détruite)	Non	Non	Plus aucun frais, tout est effacé

Deux coûts s'ajoutent au tarif horaire : la bande passante (affichée « plus bandwidth » sur l'offre) et le stockage tant que l'instance existe. Évite donc les gros téléchargements inutiles et détruis l'instance en fin de projet.

Les types de location : On-Demand, Interruptible, Reserved

Pour ce projet, choisis On-Demand : l'instance doit rester disponible pour MLflow, Airflow et Streamlit, et la différence de prix ne se compte qu'en centimes.

Type	Principe	Avantage	Risque	Convient au projet ?
On-Demand	Tarif fixe, priorité maximale : personne ne peut te reprendre la machine	Stabilité, services toujours actifs	Un peu plus cher	Oui, recommandé
Interruptible	Tu proposes un prix (enchère) :	Souvent	Coupure en	Seulement pour

Type	Principe	Avantage	Risque	Convient au projet ?
	l'instance est moins chère mais peut être mise en pause si une autre personne prend la machine	nettement moins cher	pleine exécution, attente avant redémarrage	des entraînements isolés avec checkpoints
Reserved	Engagement de durée payé à l'avance contre une remise	Prix réduit sur le long terme	Engagement, crédit bloqué	Non, inutile pour un projet de quelques jours

Si tu utilises quand même Interruptible, active `ModelCheckpoint` et `BackupAndRestore` pour reprendre l'entraînement là où il s'est arrêté, et sauvegarde régulièrement le travail en dehors de l'instance.

Configuration moyenne recommandée

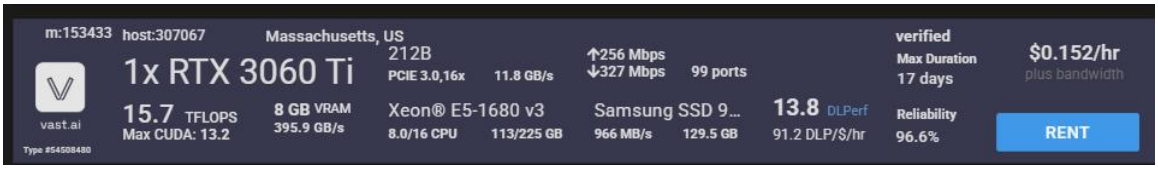
Une RTX 3060 de 12 Go avec au moins 8 vCPU et 16 Go de RAM couvre tout le projet. MobileNetV2 est un réseau léger (environ 3,5 millions de paramètres) : une carte plus puissante ne ferait gagner que quelques minutes.

Critère	Minimum conseillé	Pourquoi
GPU	RTX 3060 (12 Go) ou RTX 3090 (24 Go)	Suffisant pour du fine-tuning en 224×224
VRAM	12 Go ou plus	Évite les erreurs de mémoire avec un batch de 32 à 64
vCPU	8 ou plus	Prétraitement OpenCV et tf.data
RAM disponible	16 Go ou plus	MLflow, Airflow et Streamlit tournent en même temps
Disque	50 Go ou plus	Dataset, runs MLflow, base Airflow, modèles
Fiabilité	Supérieure à 98 %	Les services doivent rester disponibles
CUDA	12 ou plus	Compatible avec TensorFlow récent
Débit Internet	200 Mbps ou plus	Téléchargement du dataset et des paquets
Image	Template TensorFlow GPU de Vast	TensorFlow et CUDA déjà installés

Évite les cartes très récentes (RTX 50xx), car les anciennes versions de TensorFlow ne les supportent pas toujours. Pour un tarif de référence, vise une offre autour de 0,08 à 0,15 $/h pour une 3060, ou 0,18 à 0,25 $/h pour une 3090.

Lire une offre Vast.ai

L'offre RTX 3060 Ti à 0,152 dollar par heure n'est pas un bon choix pour ce projet : sa fiabilité (96,6 %) est sous le seuil conseillé, elle n'a que 8 Go de VRAM et elle coûte presque deux fois plus qu'une RTX 3060 de 12 Go.



Champ de l'offre	Ce qu'il signifie	Exemple (offre analysée)	Seuil conseillé
Prix (/hr)	Coût par heure d'allumage, bande passante en plus	0,152 dollar/h	Le plus bas possible, selon la config
GPU	Carte graphique et nombre de cartes	1x RTX 3060 Ti	1 seule carte suffit
VRAM	Mémoire de la carte graphique	8 Go	12 Go ou plus
TFLOPS	Puissance de calcul théorique	15,7	Indicatif seulement
CPU	vCPU disponibles sur le total de la machine	8,0 sur 16	8 ou plus
RAM	RAM disponible sur le total	113 sur 225 Go	16 Go ou plus
Disque	Espace de stockage de la machine	129,5 Go	50 Go ou plus
Débit réseau	Vitesses de téléchargement et d'envoi	327 et 256 Mbps	200 Mbps ou plus
Max CUDA	Version CUDA maximale supportée	13,2	12 ou plus
DLPerf	Score de performance en deep learning	13,8	Plus il est haut, plus c'est rapide
DLP par dollar	Performance par dollar dépensé (DLP/hr)	91,2	Plus il est haut, meilleur est le rapport qualité/prix
Reliability	Pourcentage de bon fonctionnement de la machine	96,6 %	Supérieur à 98 %
Max Duration	Durée maximale de location autorisée par l'hôte	17 jours	Supérieure à la durée du projet

Champ de l'offre	Ce qu'il signifie	Exemple (offre analysée)	Seuil conseillé
Verified	Machine vérifiée par Vast.ai	Oui	Oui

Pour comparer deux offres, regarde d'abord la fiabilité et la VRAM, puis le score de performance par dollar. Une RTX 3060 de 12 Go à environ 0,08 dollar/h affiche souvent plus de 140 sur ce score.

Budget : combien coûte une journée

Avec une RTX 3060 à 0,083 dollar/h, une journée entière d'allumage coûte environ 2 dollars, et une session de travail de 4 h environ 0,33 dollar. Le coût est identique que le GPU calcule ou non.

Offre	Prix/h	Session de 4 h	Journée de 8 h	24 h d'allumage
RTX 3060, 12 Go (Norvège, 99,79 % de fiabilité)	0,083	0,33	0,66	1,99
RTX 3060 Ti, 8 Go (offre analysée)	0,152	0,61	1,22	3,65
RTX 3090, 24 Go (Californie)	0,180	0,72	1,44	4,32
RTX 3090, 24 Go (Estonie)	0,252	1,01	2,02	6,05

Tous les montants sont en dollars, hors bande passante et stockage, qui ajoutent quelques centimes.

Pour ne jamais dépasser un budget par jour, divise-le par 24 : un plafond de 3 dollars par jour correspond à une offre à 0,125 dollar/h ou moins, même si l'instance reste allumée toute la journée.

Le projet complet (3 entraînements, MLflow, Streamlit et Airflow) devrait coûter 2 à 3 dollars si l'instance est arrêtée dès qu'on ne s'en sert plus. Un crédit de 5 dollars est largement suffisant.

Allouer, utiliser, arrêter, détruire

Le cycle complet tient en sept étapes : créer le compte, charger le crédit, filtrer, louer, travailler, arrêter, détruire.

1. Crée un compte sur Vast.ai et charge un crédit de 5 dollars, suffisant pour tout le projet.
2. Ouvre l'onglet de recherche (Search) et choisis le template TensorFlow GPU de Vast.
3. Filtre les offres : 1 GPU, mode On-Demand, RTX 3060 ou 3090, fiabilité supérieure à 98 %, disque d'au moins 50 Go, CUDA 12 ou plus.
4. Trie par prix croissant et compare la VRAM, la RAM disponible, les vCPU et la fiabilité (voir la section sur la lecture d'une offre).
5. Clique sur RENT et ouvre l'instance depuis l'onglet Instances : le bouton Open lance Jupyter avec le token déjà inclus.
6. Travaille, puis clique sur Stop dès que tu fais une pause : le GPU n'est plus facturé.
7. À la fin du projet, sauvegarde tes fichiers (code, modèles, runs MLflow) sur GitHub ou en local, puis clique sur Destroy pour arrêter aussi les frais de stockage.

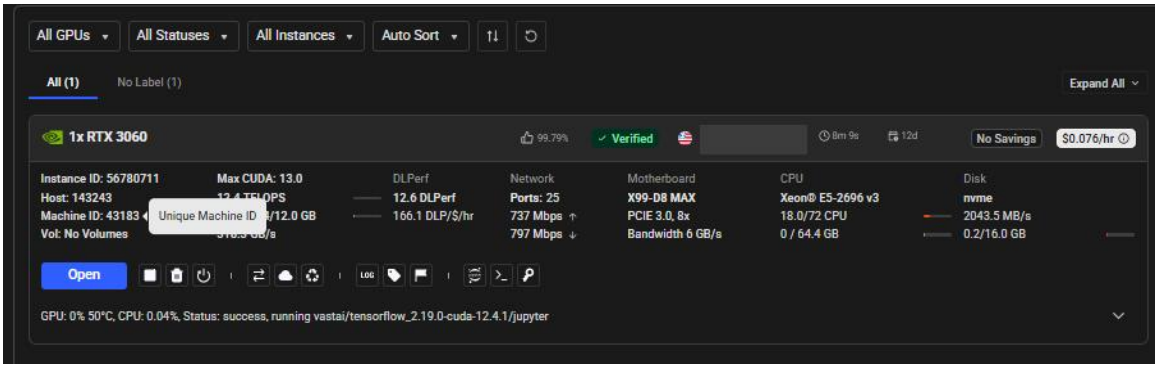
Pour accéder aux interfaces depuis ton navigateur, utilise un tunnel SSH ou les ports redirigés par Vast : MLflow (5000), Streamlit (8501) et Airflow (8080).

Se connecter à la VM

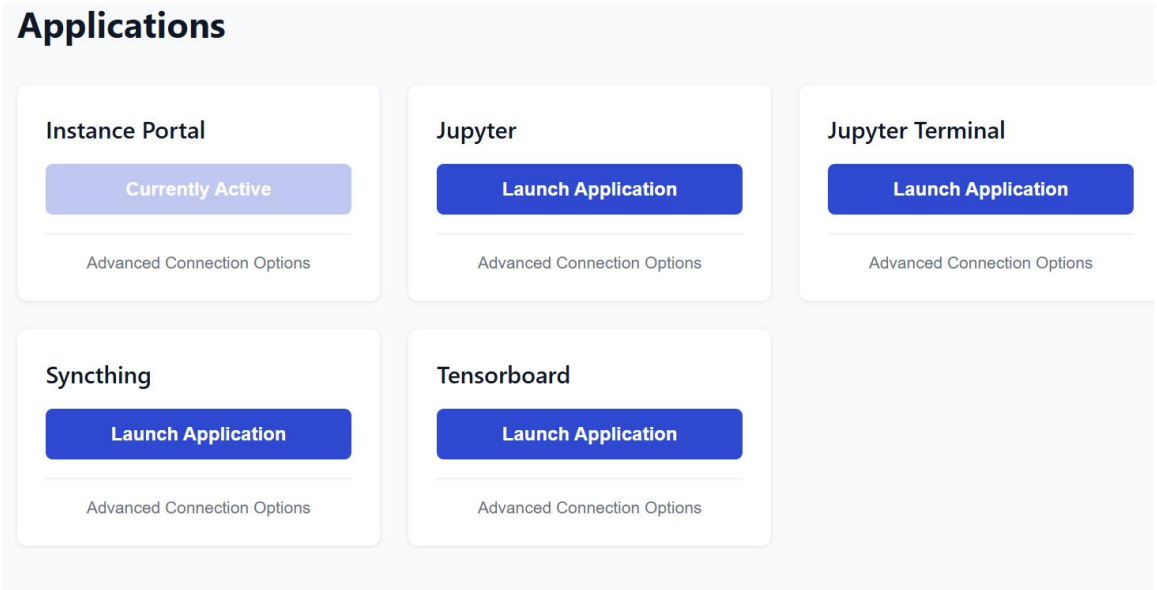
Deux méthodes permettent de travailler sur l'instance : Jupyter avec VS Code, la plus simple (un lien et un token), et SSH avec VS Code, plus complète, car elle redirige aussi les ports de MLflow, Streamlit et Airflow. Dans les deux cas, l'instance doit être démarrée (statut Running).

Méthode 1 : Jupyter et VS Code

Étape 1 : démarrer l'instance. Dans l'onglet Instances, vérifie que l'instance sélectionnée est démarrée (statut Running).

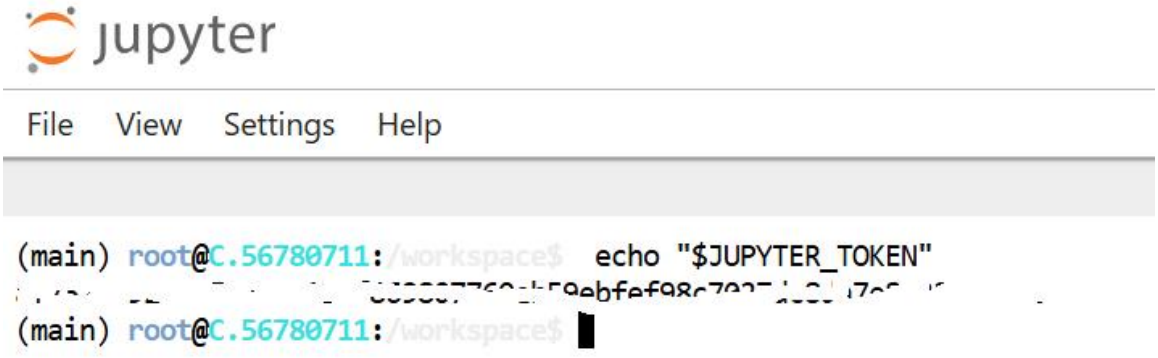


Étape 2 : cliquer sur le bouton Open. La page Applications s'affiche : Instance Portal, Jupyter, Jupyter Terminal, Syncthing et Tensorboard.

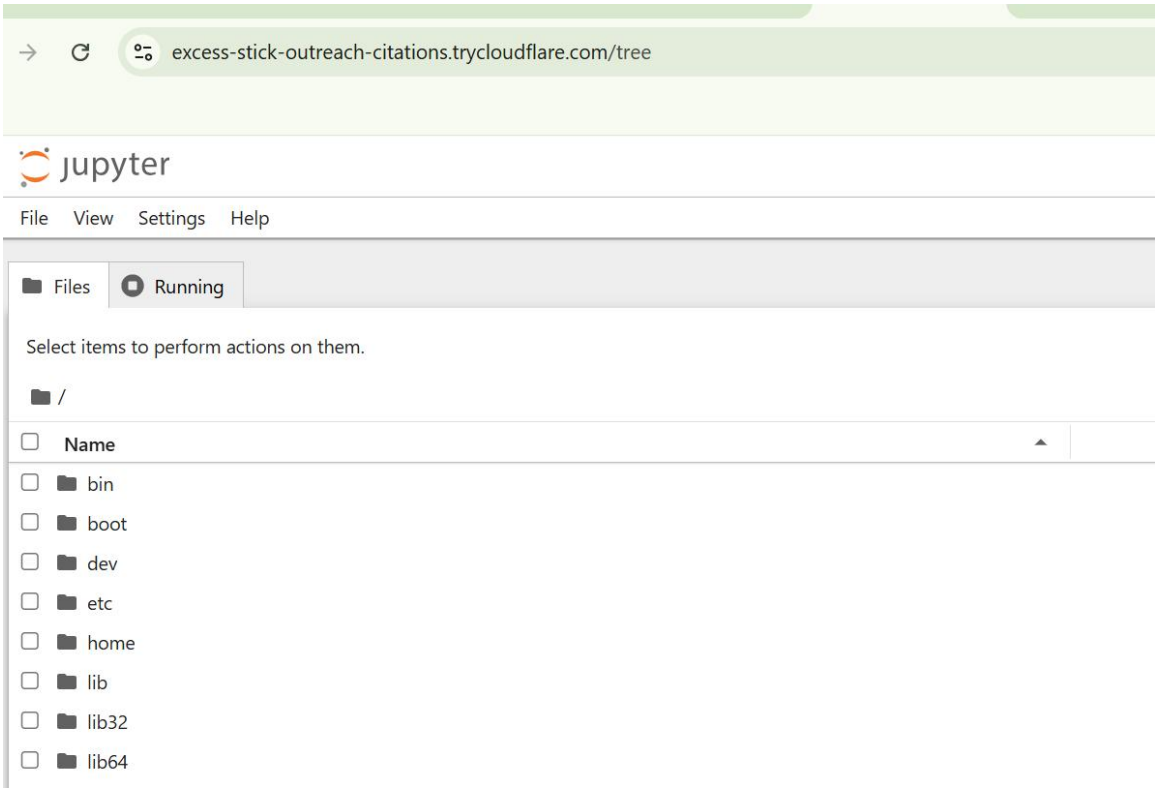


Étape 3 : ouvrir la ligne de commande du serveur Jupyter. Clique sur Launch Application sous Jupyter Terminal.

Étape 4 : récupérer le token. Dans ce terminal, exécute la commande `echo "$JUPYTER_TOKEN"`. Le token est la chaîne de caractères affichée sous la commande.

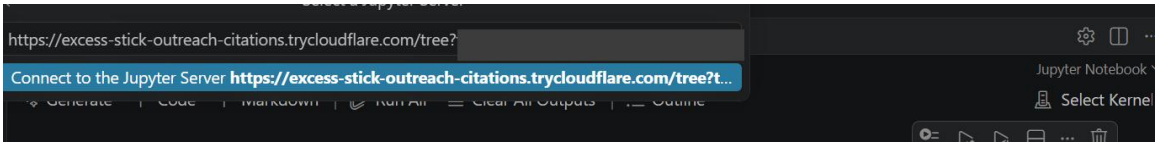


Étape 5 : ouvrir le serveur Jupyter et copier son lien. Reviens à la page Applications, clique sur Launch Application sous Jupyter, puis copie le lien affiché dans la barre d'adresse du navigateur.



Étape 6 : ajouter le token au lien. Ajoute `?token=` suivi du token à la fin du lien, ce qui donne `https://adresse-du-serveur/tree?token=ton_token`.

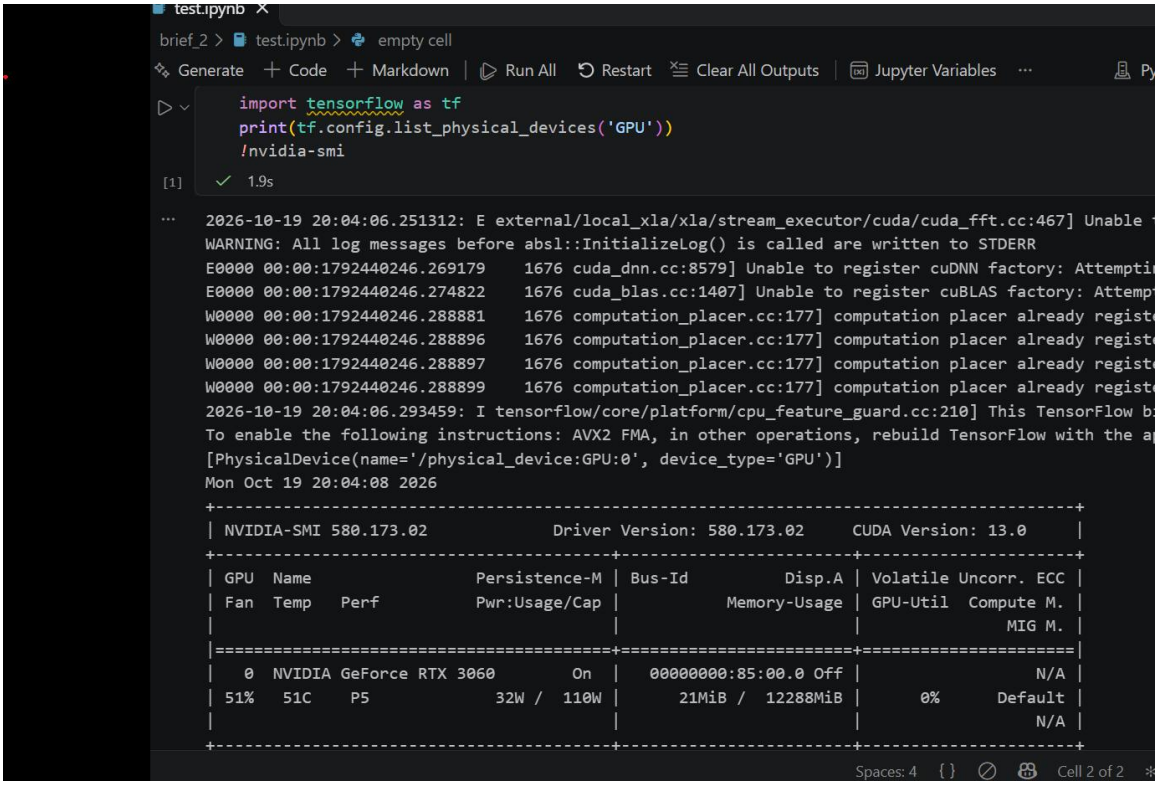
Étape 7 : connecter VS Code au serveur. Ouvre un notebook (.ipynb), clique sur Select Kernel, choisis Existing Jupyter Server, colle le lien avec le token, puis valide avec Connect to the Jupyter Server.



Étape 8 : vérifier la présence du GPU. Exécute ce code dans une cellule du notebook :

```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
!nvidia-smi
```

Le résultat attendu est une ligne contenant `PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')`, puis un tableau nvidia-smi qui affiche le modèle de la carte (par exemple RTX 3060 avec 12288 Mo de mémoire). Les avertissements TensorFlow sur cuDNN ou cuBLAS au démarrage sont fréquents : l'important est que le GPU soit listé.



Méthode 2 : SSH avec VS Code

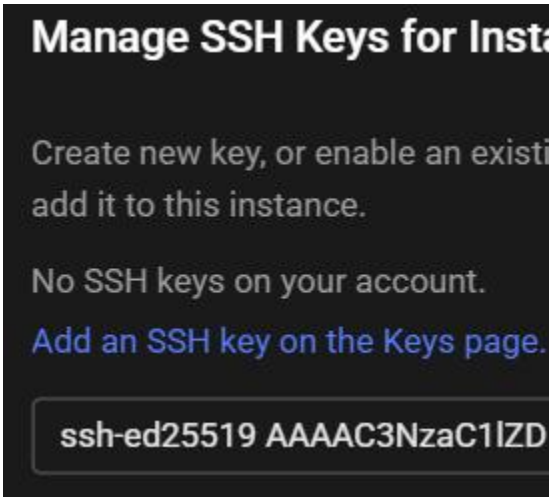
Étape 1 : générer la paire de clés. Dans l'invite de commandes Windows, exécute la commande ci-dessous, sans passphrase.

```
ssh-keygen -t ed25519 -C "vast" -f "%USERPROFILE%\.ssh\vast_key" -N ""
```

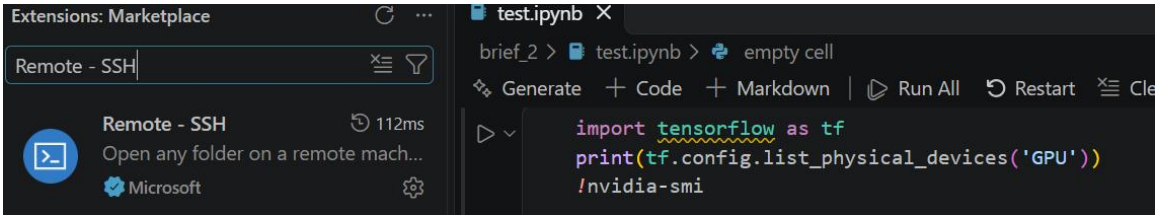
Deux fichiers sont créés dans le dossier .ssh : `vast_key` (clé privée, à ne jamais partager) et `vast_key.pub` (clé publique, à donner à Vast.ai).

```
C:\Users\Simplon>ssh-keygen -t ed25519 -C "vast" -f "%USERPROFILE%\.ssh\vast_key" -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in C:\Users\Simplon\.ssh\vast_key
Your public key has been saved in C:\Users\Simplon\.ssh\vast_key.pub
The key fingerprint is:
```

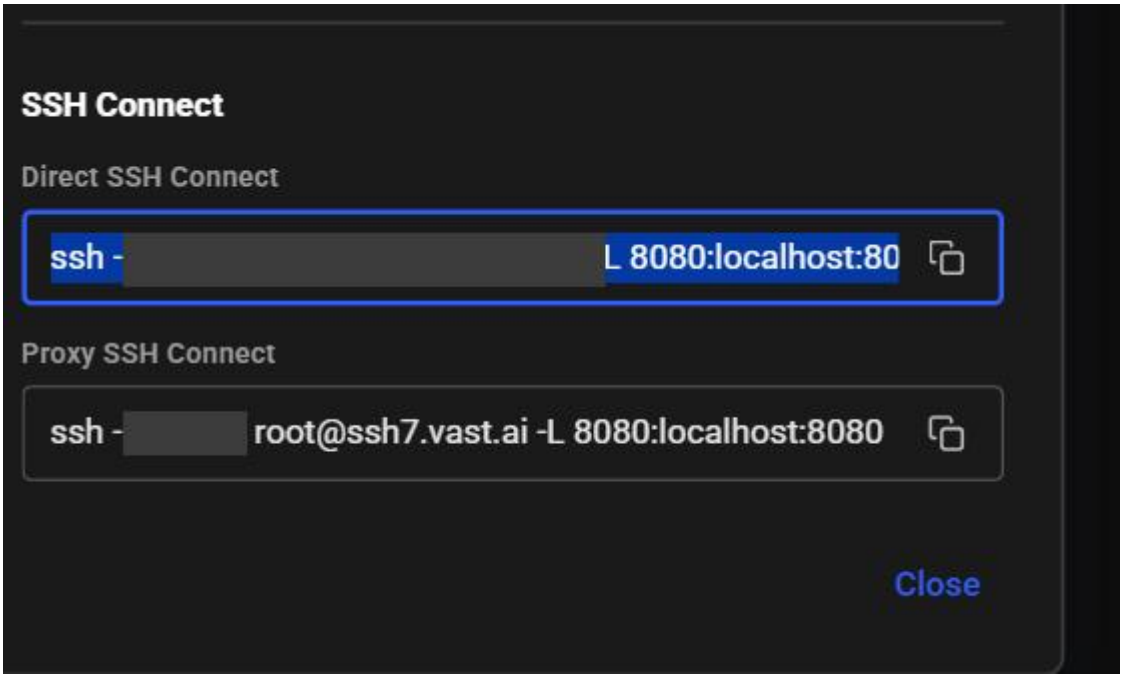
Étape 2 : ajouter la clé publique sur Vast.ai. Affiche le contenu du fichier avec `type "%USERPROFILE%\.ssh\vast_key.pub"`, copie-le, puis colle-le dans la page Keys du compte Vast.ai ou dans la fenêtre Manage SSH Keys de l'instance. Cette clé commence par `ssh-ed25519`.



Étape 3 : installer l'extension Remote - SSH. Dans VS Code, ouvre l'onglet Extensions, recherche Remote - SSH (éditeur Microsoft) et installe-la.



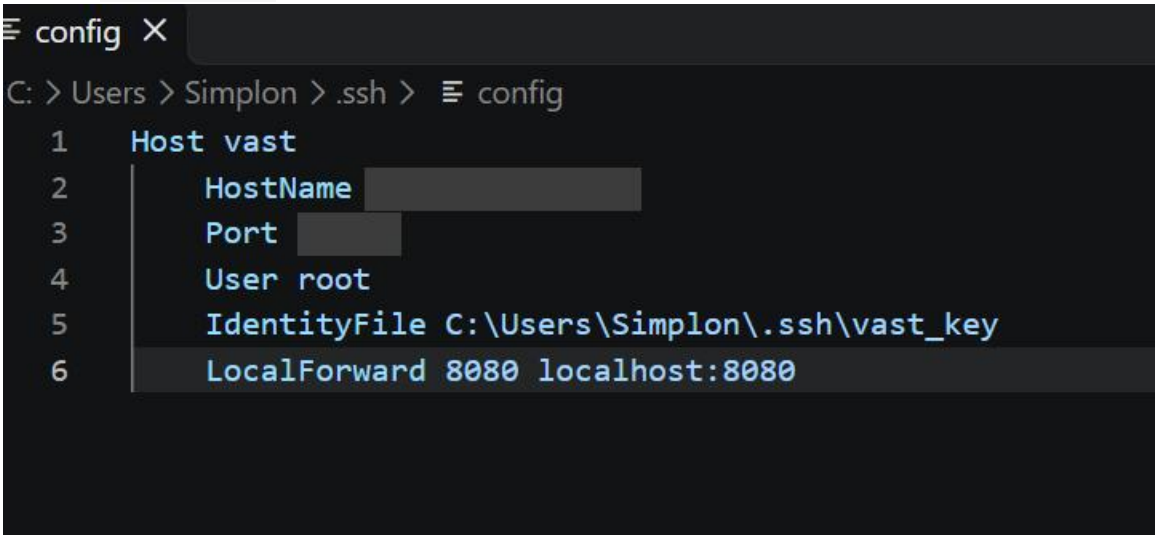
Étape 4 : récupérer l'adresse SSH de l'instance. Sur la carte de l'instance, ouvre la fenêtre SSH Connect. Dans Direct SSH Connect, repère l'adresse IP (après `root@`) et le port (après `-p`). Proxy SSH Connect est une solution de secours si la connexion directe échoue. L'IP et le port changent quand une nouvelle instance est créée : relis-les à chaque fois.



Étape 5 : remplir le fichier config. Dans VS Code, ouvre le fichier `C:\Users\ton_nom\.ssh\config` et ajoute ce bloc en remplaçant les valeurs entre chevrons.

```
Host vast
    HostName ip_de_l_instance
    Port port_ssh
    User root
    IdentityFile C:\Users\ton_nom\.ssh\vast_key
    LocalForward 8080 localhost:8080
```

La ligne `LocalForward` redirige un port de l'instance vers ton ordinateur : 8080



Étape 6 : ouvrir la connexion. Dans VS Code, ouvre la palette de commandes (Ctrl+Maj+P), choisis Remote-SSH: Connect to Host, sélectionne `vast`, puis ouvre le dossier `/workspace`. Vérifie le GPU en tapant `nvidia-smi` dans le terminal de VS Code.

Checklist pour ne pas gaspiller le crédit

La règle principale : une instance allumée coûte toujours le même prix, donc elle ne doit tourner que pendant que tu l'utilises.

- ☐ Préparer et tester le code sur une petite partie des données, en local ou sur Google Colab, avant de louer le GPU.
- ☐ Ne louer qu'une seule instance à la fois, avec une offre au bon rapport qualité/prix (RTX 3060 ou 3090, fiabilité supérieure à 98 %).
- ☐ Cliquer sur Stop à chaque pause, même courte, et ne jamais laisser l'instance tourner la nuit.
- ☐ Activer `ModelCheckpoint` et `BackupAndRestore` pour ne pas perdre un entraînement en cas de coupure.
- ☐ Sauvegarder régulièrement le code, les modèles et les runs MLflow sur GitHub ou en local.
- ☐ Éviter les gros téléchargements inutiles, car la bande passante est facturée à part.
- ☐ Vérifier le solde dans la console Vast.ai à chaque début de session.
- ☐ Détruire l'instance à la fin du projet pour arrêter aussi les frais de stockage.